

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

PROGRAMACIÓN AVANZADA



Tarea: Implementación de una Solución Robusta

NRC: 30405

Docente:

Paulo César Galarza Sánchez

NOMBRE:

Fabricio Fernanda Baquero López

19/05/2026

Implementación de una Solución Robusta

1. Objetivo del Reto

El presente reto tuvo como finalidad desarrollar una API REST funcional utilizando Spring Boot 4.x y Java 21, aplicando de manera adecuada el patrón de arquitectura por capas y la configuración de perfiles de entorno (Dev y Prod). Esta implementación permitió simular un entorno de desarrollo profesional, incorporando buenas prácticas relacionadas con organización del código, mantenimiento y administración de configuraciones según el ambiente de ejecución.

2. Diagrama de Paquetes

La solución fue estructurada mediante una arquitectura por capas, permitiendo dividir las responsabilidades de la aplicación en componentes independientes y organizados. Esta metodología favorece la mantenibilidad, escalabilidad y reutilización del código, aspectos fundamentales en el desarrollo de aplicaciones empresariales con Spring Boot.

La estructura de paquetes implementada en IntelliJ IDEA permitió separar adecuadamente la capa de controladores, servicios, repositorios y configuraciones, facilitando tanto el desarrollo como futuras ampliaciones del sistema.

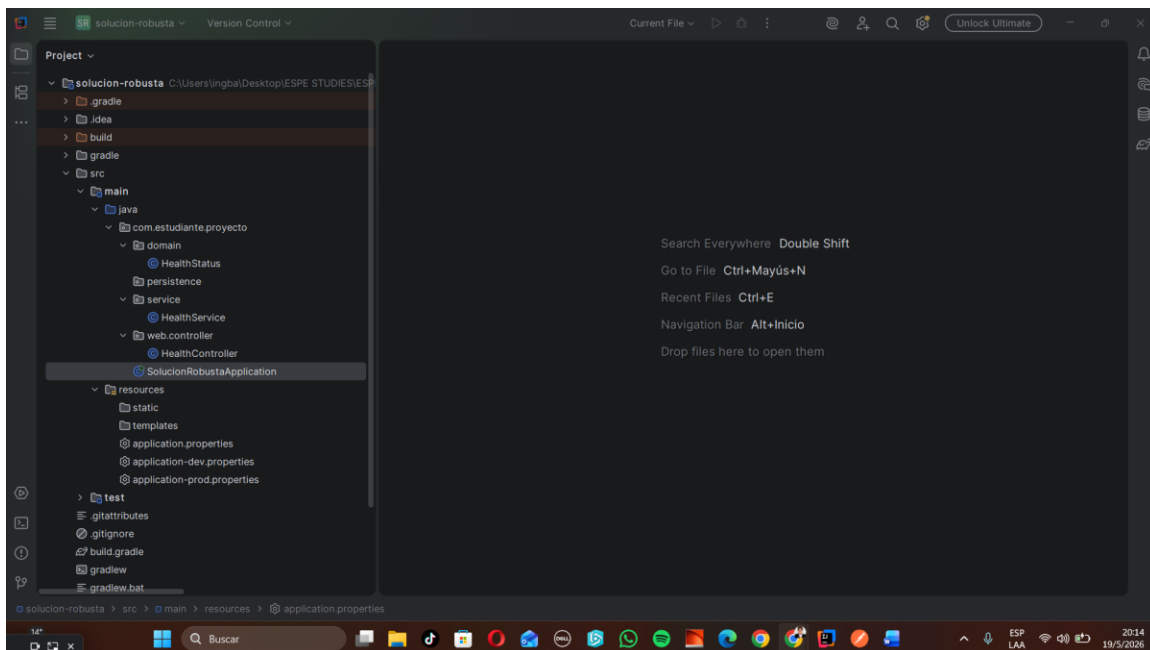


Figura 1. Estructura de paquetes implementada en IntelliJ IDEA utilizando arquitectura por capas.

3. Configuración de Entornos

La aplicación incorporó perfiles de configuración mediante Spring Profiles, permitiendo modificar dinámicamente el comportamiento del sistema dependiendo del entorno en el que se ejecute. Para ello, se implementaron los perfiles Dev y Prod, simulando escenarios reales de desarrollo y producción.

Perfil	Puerto	Logs	Finalidad
Dev	8080	DEBUG	Desarrollo y pruebas
Prod	9090	ERROR	Producción y despliegue

- ***application.properties***

El archivo `application.properties` fue utilizado como configuración principal de la aplicación, estableciendo el perfil activo por defecto y parámetros generales de la API REST.

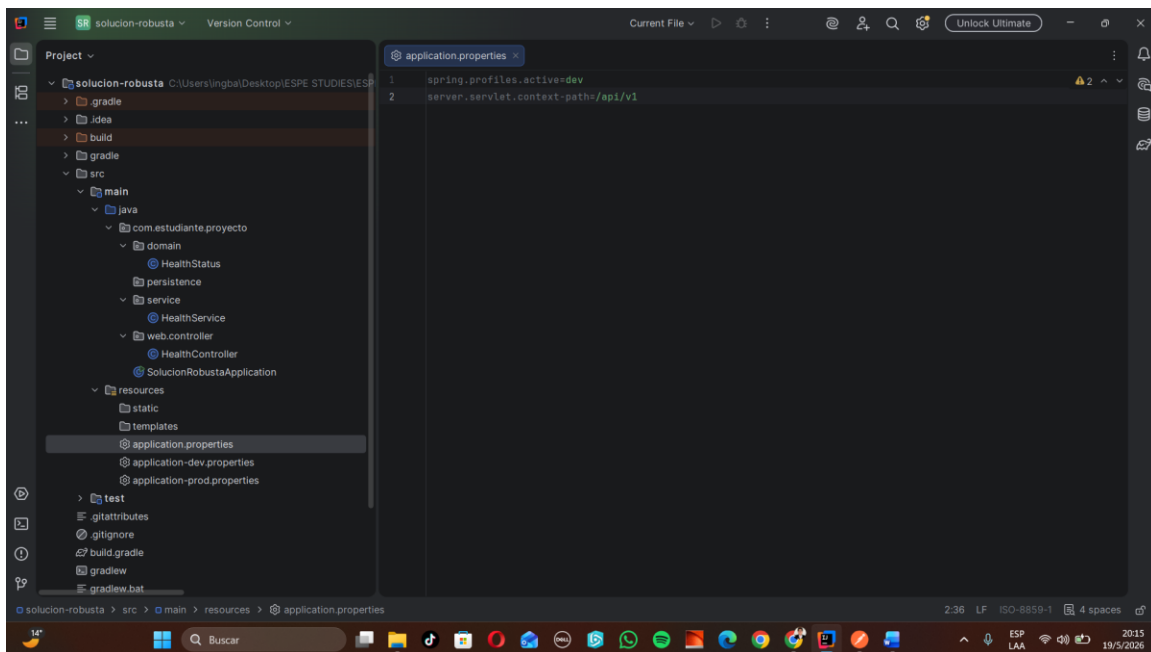


Figura 2. Configuración principal de la aplicación mediante el archivo `application.properties`.

- ***application-dev.properties***

El entorno de desarrollo fue configurado utilizando el puerto 8080 y registros de tipo DEBUG, permitiendo obtener información detallada durante las pruebas y el monitoreo del sistema.

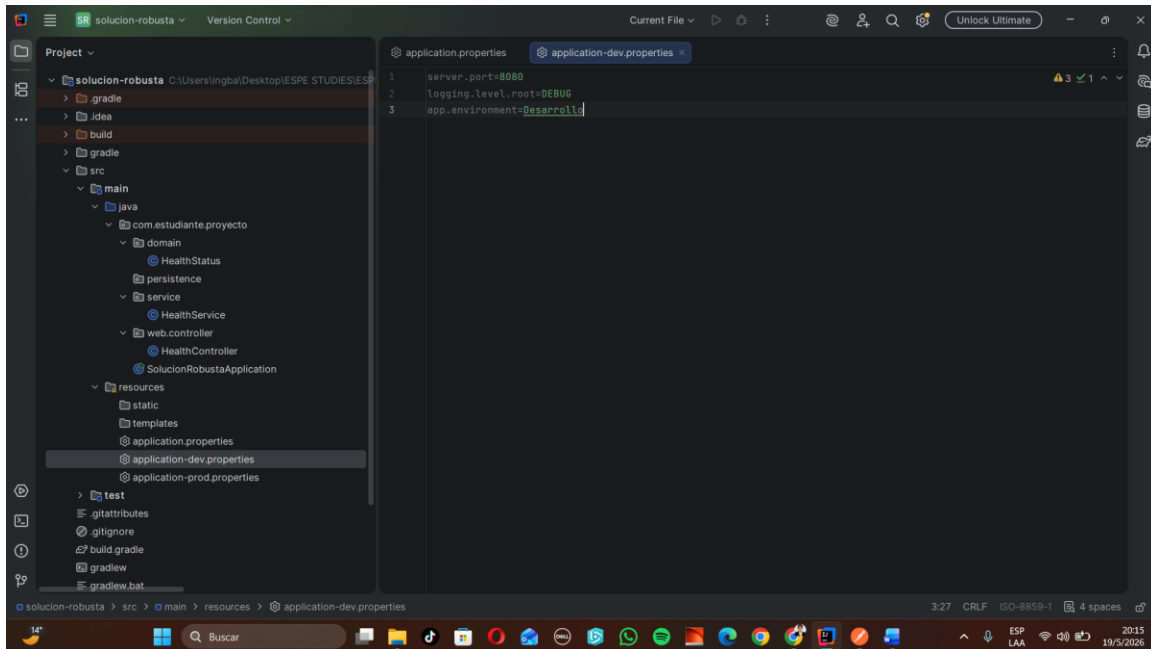


Figura 3. Configuración del entorno Dev utilizando el puerto 8080 y logs DEBUG.

- ***application-prod.properties***

Para el entorno de producción se configuró el puerto 9090 y un nivel de logs restringido únicamente a mensajes de tipo ERROR, simulando un ambiente más controlado y orientado al despliegue profesional.

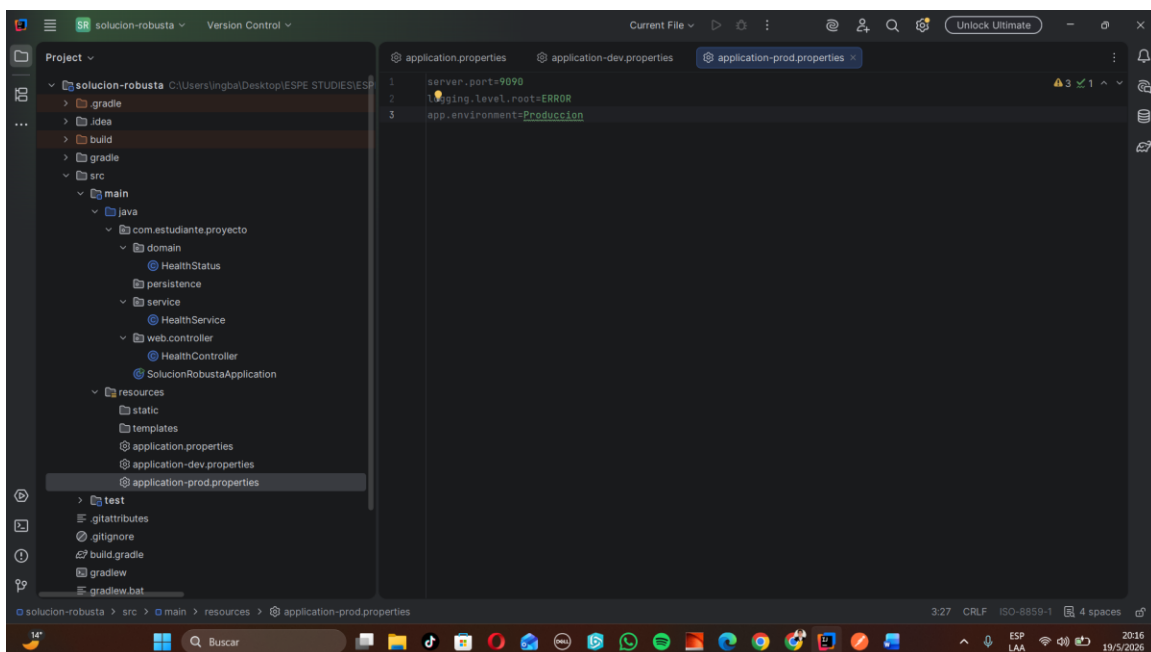


Figura 4. Configuración del entorno Prod utilizando el puerto 9090 y logs ERROR.

La implementación de perfiles mediante Spring Profiles permitió separar configuraciones específicas según el entorno de ejecución de la aplicación. El perfil Dev estuvo orientado al desarrollo y pruebas locales, proporcionando mayor nivel de detalle en los registros del sistema. En contraste, el perfil Prod fue diseñado para un escenario de producción, limitando los logs a errores críticos y utilizando un puerto diferente para simular un entorno empresarial real. Esta estrategia contribuye significativamente a mejorar la administración, mantenimiento y escalabilidad de aplicaciones modernas.

3.1 Configuración Técnica del Proyecto

El proyecto fue desarrollado utilizando Gradle como herramienta de automatización y gestión de dependencias. Asimismo, se estableció Java 21 como versión principal del entorno de desarrollo y Spring Boot 4.0.6 como framework base para la construcción de la API REST.

Esta configuración permitió integrar de manera eficiente las dependencias necesarias, optimizar la administración del proyecto y garantizar compatibilidad con tecnologías modernas del ecosistema Java.

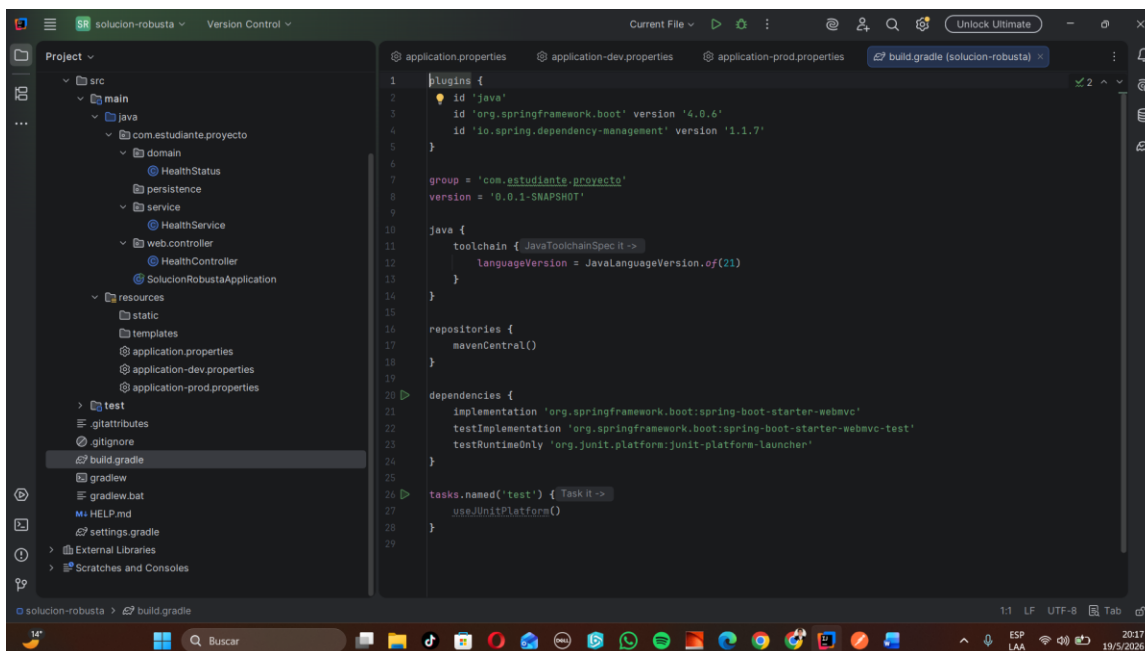


Figura 5. Configuración técnica del proyecto utilizando Gradle, Spring Boot 4.0.6 y Java 21.

4. Pruebas de Funcionamiento

Las pruebas funcionales fueron realizadas mediante Postman con el objetivo de verificar el correcto comportamiento del endpoint REST implementado en ambos perfiles de configuración. Estas validaciones permitieron comprobar la conectividad, la respuesta del sistema y el cambio dinámico de entornos según el perfil activo.

4.1 Prueba del Entorno DEV

La aplicación fue ejecutada utilizando el perfil Dev, configurado con el puerto 8080 y registros de depuración tipo DEBUG. Mediante Postman se efectuó una solicitud GET al endpoint `/api/v1/health`, obteniendo una respuesta satisfactoria con código HTTP 200 OK, evidenciando además el perfil activo correspondiente.

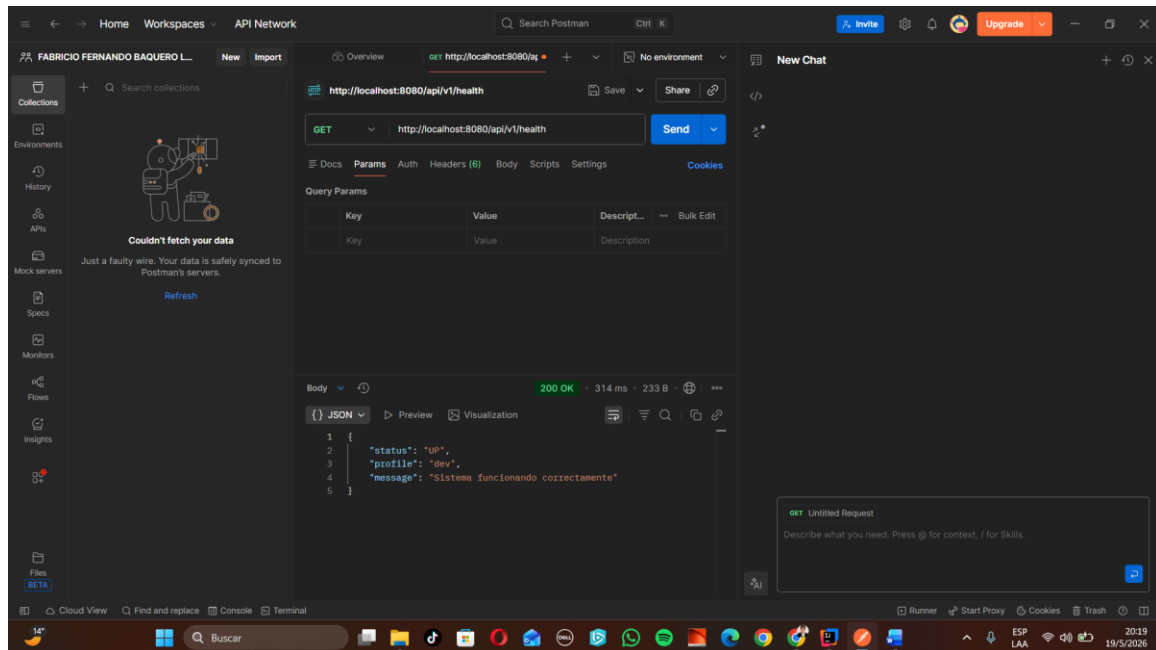


Figura 6. Validación funcional del endpoint REST utilizando el perfil Dev en el puerto 8080 mediante Postman.

4.2 Prueba de Entorno Prod

Posteriormente, la aplicación fue ejecutada utilizando el perfil Prod, configurado con el puerto 9090 y registros limitados únicamente a errores críticos. A través de Postman se realizó nuevamente una solicitud GET al endpoint `/api/v1/health`, obteniendo una respuesta exitosa con código HTTP 200 OK y confirmando el correcto funcionamiento del perfil de producción.

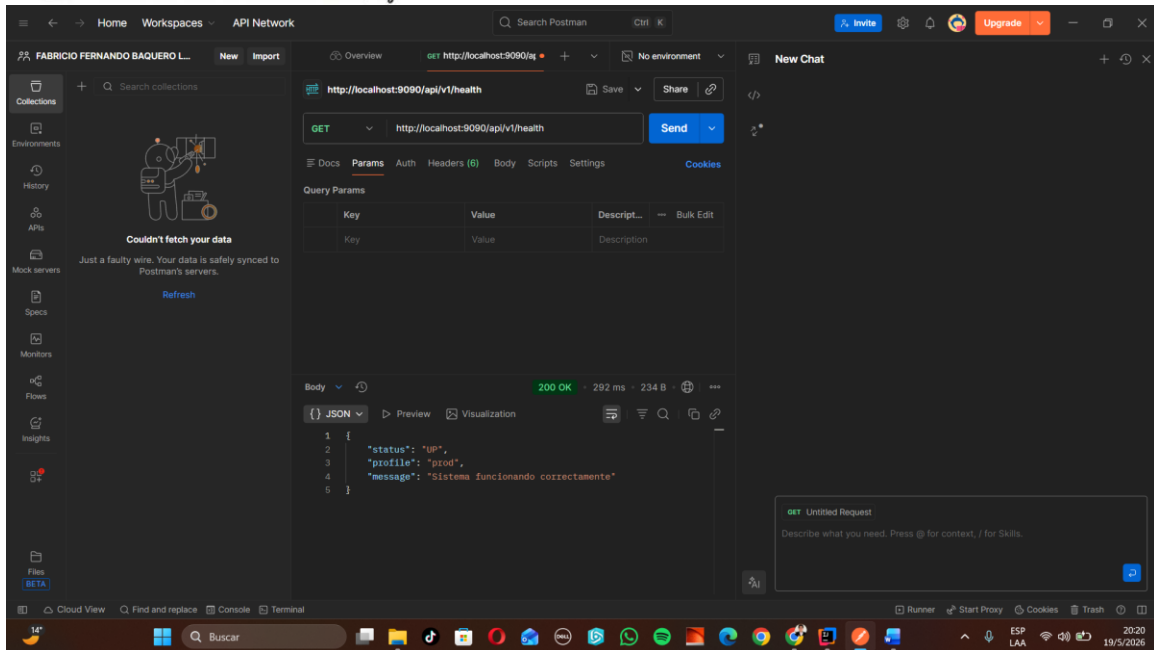


Figura 7. Validación funcional del endpoint REST utilizando el perfil Prod en el puerto 9090 mediante Postman.

5. Análisis de Resultados

Los resultados obtenidos durante las pruebas permitieron comprobar el funcionamiento adecuado de la arquitectura implementada y de la gestión dinámica de perfiles mediante Spring Boot. La API respondió correctamente tanto en el entorno Dev como en Prod, demostrando que la configuración basada en perfiles permite modificar parámetros específicos de ejecución sin afectar la lógica principal del sistema. Además, se evidenció que la utilización de perfiles facilita la administración de configuraciones dependiendo del ambiente de trabajo, práctica ampliamente utilizada en el desarrollo de aplicaciones empresariales modernas.

6. Conclusiones

1. La implementación de una arquitectura por capas permitió distribuir de manera adecuada las responsabilidades de la aplicación, mejorando la organización del código y facilitando el mantenimiento del sistema.
2. El uso de Spring Profiles posibilitó la configuración dinámica de los entornos Dev y Prod, permitiendo adaptar el comportamiento de la aplicación según las necesidades del entorno de ejecución.
3. La integración de Spring Boot con Gradle y Java 21 facilitó el desarrollo de la API REST, proporcionando un entorno moderno, eficiente y alineado con buenas prácticas de desarrollo empresarial.
4. Las pruebas realizadas mediante Postman validaron el correcto funcionamiento del endpoint REST en ambos perfiles configurados, confirmando la estabilidad de la solución implementada.
5. La arquitectura por capas constituye una estrategia fundamental para el desarrollo de aplicaciones escalables y mantenibles, ya que favorece la reutilización de componentes y simplifica futuras ampliaciones o integraciones del sistema.